

Bitácoras Sistemas Embebidos Primer Corte

Julian Díaz Cortina
Ingeniería Mecatrónica
Universidad Santo Tomás
Bucaramanga, Colombia

juliandavid.diaz@ustabuca.edu.co

Fabian Ruiz Mateus
Ingeniería Mecatrónica
Universidad Santo Tomás
Bucaramanga, Colombia

fabianaugusto.ruiz@ustabuca.edu.co

Abstract—This document is a template in Spanish for writing articles in LaTeX using the IEEE format. It briefly explains the objective of the work, the general methodology, and the main results obtained.

Index Terms—LaTeX, IEEEtran, templates, figures, tables, bibliography.

RESUMEN

Este documento es una plantilla en español para escribir artículos en LaTeX con el formato IEEE. Aquí se explica brevemente el objetivo del trabajo, la metodología general y el resultado principal.

PALABRAS CLAVE

LaTeX, IEEEtran, plantillas, figuras, tablas, bibliografía.

I. INTRODUCCIÓN

En los artículos originales enviados a la revista IEEE, la introducción debe presentar de forma concisa el contexto y la relevancia del problema abordado, describiendo brevemente el área de estudio y su importancia en el campo de la ingeniería o tecnología; incluir un resumen del estado del arte con referencia a los trabajos más relevantes y recientes, destacando las limitaciones o vacíos existentes que motivan la investigación; exponer con claridad el objetivo y la principal contribución del trabajo, indicando qué se propone y en qué se diferencia o mejora respecto a lo ya publicado; y, opcionalmente, cerrar con una breve descripción de la estructura del artículo para guiar al lector en el desarrollo del contenido.

La escritura académica en LaTeX ofrece control tipográfico y reproducibilidad. Esta plantilla en español muestra, con ejemplos mínimos, cómo estructurar un artículo usando IEEEtran: figuras, tablas, ecuaciones y bibliografía. Úsala para practicar el flujo básico de edición y compilación.

El resto del documento se organiza así: en la Sección II se describe un ejemplo de desarrollo experimental simplificado; en la Sección III se explica la metodología con ecuaciones de ejemplo; en la Sección VII se muestran tablas y figuras de ejemplo; y en la Sección VIII se listan conclusiones y trabajo futuro.

LaTeX facilita gestionar referencias con BibTeX, por ejemplo [? ?]. También puedes usar estilos numéricos compatibles con IEEE. Las citas deben registrarse en el archivo `reference.bib`

Diversos trabajos describen técnicas de análisis espectral y aprendizaje automático aplicadas a señales [? ?]. Ajusta esta sección a tu tema específico.

II. MARCO REFERENCIAL

A. Actividad 1

1) *Caso de estudio*: Se desea diseñar un sistema basado en microcontrolador que:

- Lea un sensor cada 10 ms.
- Controle un motor según la lectura.
- Envíe datos por comunicación serial cada 100 ms.
- Atienda un botón de emergencia que debe detener todo inmediatamente.
- Muestre información en una pantalla.

• **¿Cómo implementar esto usando solo un while?:** La respuesta de esta pregunta genera como resultado el diagrama de flujo que se encuentra en la figura 1.

• **¿Qué problemas podrían aparecer?:** Al ser un while con dependencia doble, interna y distinta del tiempo; se puede estar presentando retrasos de lectura, pues la lectura lineal debe esperar a, por ejemplo, enviar el paquete de datos por el puerto serial para continuar con el proceso. Esto mismo trae problemas de latencia y de estabilidad, pues en el caso que se desee agregar mas tareas, el control del tiempo se verá aún mas afectado.

• **¿Qué tarea sería más crítica?:** En este caso desarrollado, se entiende que la tarea más crítica sería toda la que conlleve al botón de emergencia, pues estamos hablando de factores de seguridad humana y estos deben ser considerados de manera prioritaria.

• **¿Cómo garantizarías tiempos de respuesta?:** En una primera parte, se deberían independizar dos elementos claves: La parada de emergencia y el timer, pues estos deben ser independientes de una revisión ocasional dentro del while. Una manera de hacerlo es generando interrupciones; asegurando una baja latencia y por lo tanto (en el caso de la parada de emergencia) un menor tiempo de reacción. Por parte del Timer; considero que se debería tratar como variable independiente donde dicha variable sea la única que gestione la lectura y el envío de datos con

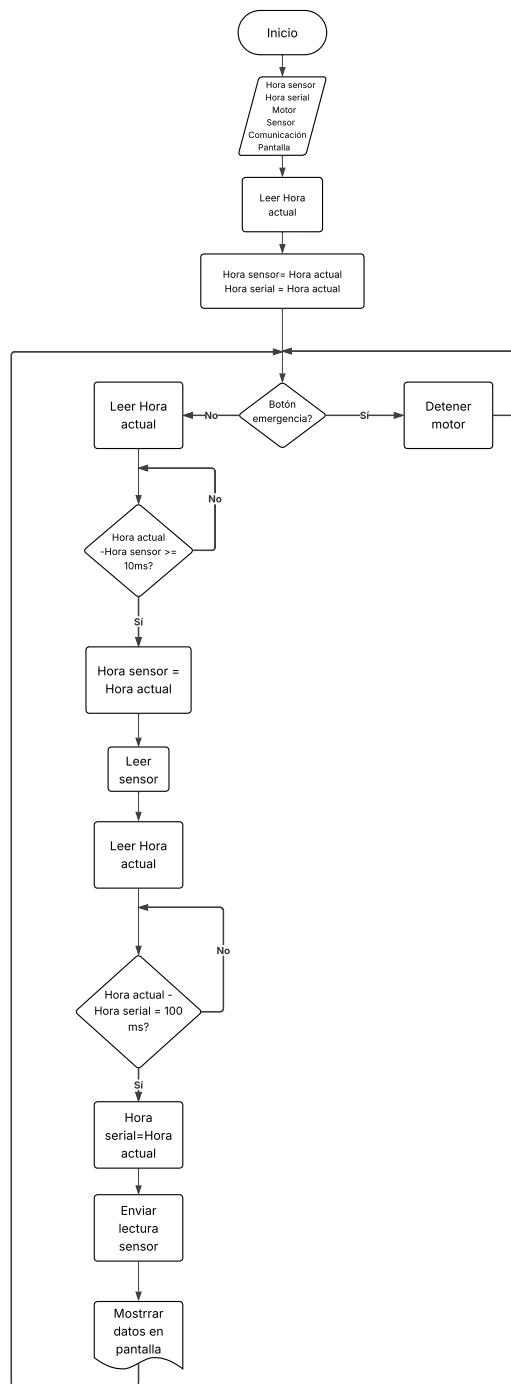


Figure 1. Diagrama de flujo de la implementación del while

el fin de asegurar un "espacio de memoria" que realmente cumpla los tiempos establecidos.

2) Trabajo de consulta:

- **Interrupciones:** Las interrupciones son mecanismos que permiten a un microcontrolador suspender temporalmente la ejecución del programa para atender un evento inmediato. Cuando ocurre dicho evento, el hardware guarda el

estado del procesador, ejecuta una Rutina de Servicio de Interrupción (*Interrupt Service Routine-ISR*) y luego restaura el estado para continuar la ejecución normal.

Su función principal es evitar el *polling* (método en el que el procesador revisa continuamente si ocurrió un evento, en lugar de esperar a que una interrupción lo notifique) constante y mejorar el tiempo de respuesta ante eventos internos o externos, siendo esenciales en sistemas de tiempo real. [?] No obstante, pueden generar condiciones de carrera, aumento de latencia si la ISR es extensa y problemas de memoria de pila, por lo que deben implementarse de forma breve y controlada. [?]

- **Planificación y Scheduler:** El scheduler (planificador) es el componente del sistema operativo o del software de control que decide qué tarea se ejecuta y en qué momento. En sistemas embebidos con multitarea, su función es administrar el uso del procesador cuando existen varias tareas que requieren ejecución, garantizando orden y control en el acceso al CPU (*Central Processing Unit*). [?] La prioridad es el nivel de importancia asignado a cada tarea. Determina cuál debe ejecutarse primero cuando varias están listas al mismo tiempo... Una tarea con mayor prioridad desplaza a otra de menor prioridad si el sistema lo permite. La planificación preventiva (preemptiva) es aquella en la que el scheduler puede interrumpir una tarea en ejecución para asignar el procesador a otra de mayor prioridad. En cambio, la planificación cooperativa requiere que la tarea en ejecución ceda voluntariamente el control del procesador, por lo que el cambio de tarea ocurre solo cuando esta finaliza o libera el CPU. [?]

- **Multitarea:** La concurrencia es la capacidad de un sistema para gestionar múltiples tareas que progresan en intervalos de tiempo superpuestos. No implica necesariamente que todas se ejecuten exactamente al mismo tiempo, sino que el sistema organiza su ejecución de manera que todas avancen sin bloquearse mutuamente. [?] La multitarea real ocurre cuando varias tareas se ejecutan simultáneamente en diferentes núcleos de procesamiento. En cambio, la multitarea simulada se presenta en sistemas con un solo núcleo, donde el procesador alterna rápidamente entre tareas mediante un scheduler, dando la impresión de ejecución simultánea aunque solo una tarea se ejecuta en cada instante.

En sistemas embebidos, ejemplos de multitarea incluyen: un microcontrolador que controla un motor mientras lee sensores y transmite datos por comunicación serial; un sistema IoT (*Internet of Things*) que adquiere datos, los procesa y los envía a la nube; o un dispositivo médico que monitorea señales mientras actualiza una pantalla. En todos estos casos, las tareas comparten el procesador y requieren coordinación adecuada. [?]

- **Sincronización:** Una condición de carrera es una situación que ocurre cuando dos o más tareas acceden

y modifican un mismo recurso o variable compartida de manera simultánea, y el resultado final depende del orden impredecible en que se ejecuten [?]. Esto puede generar inconsistencias o corrupción de datos si no existen mecanismos de control adecuados.

Los semáforos y los mutex son mecanismos de sincronización utilizados para coordinar el acceso a recursos compartidos. Un semáforo es una variable de control que regula la disponibilidad de un recurso y puede permitir uno o varios accesos según su tipo. Un mutex (mutual exclusion) es un mecanismo específico que garantiza que solo una tarea a la vez pueda acceder a un recurso, asegurando exclusión mutua.

Una sección crítica es el fragmento de código en el que se accede a un recurso compartido y que, por lo tanto, debe ejecutarse de forma exclusiva para evitar interferencias entre tareas. Para protegerla, se emplean mecanismos como mutex, semáforos o des-habilitación temporal de interrupciones. [?]

III. PRACTICAS

A. Reto 1

Diseñar un sistema que permita activar una cerradura electrónica si se cumple una condición lógica (contraseña digital o combinación de botones). Requisitos obligatorios:

- Entrada digital (botones o teclado)
- Salida: LED indicador + actuador (servo o relé)
- Manejo de intentos fallidos
- Temporizador de bloqueo tras múltiples errores
- Modelado opcional como máquina de estados

1) *Análisis del problema:*

- **Identificación de entradas y salidas:** Según los requisitos obligatorios del sistema, se identifica la necesidad de tener una entrada digital correspondiente, en este caso; a tres botones que ejecutarán la secuencia combinacional para la apertura de la cerradura. En lo que corresponde a la salida, se usa un Diodo LED como indicador lumínico de apertura de la cerradura que será representada por el funcionamiento de un servomotor.
- **Restricciones técnicas:** El sistema presenta restricciones eléctricas relacionadas con los niveles de tensión, corriente máxima de los pines GPIO y los requerimientos de alimentación del actuador.
 - El sistema presenta unos límites eléctricos correspondientes a sus 3 elementos principales, ESP32, Servomotor y Led. El microcontrolador ESP32 opera con niveles lógicos de 3.3[V] en sus pines GPIO que pueden suministrar corrientes limitadas (12 mA recomendados [?]) . El servomotor requiere una alimentación de aproximadamente 5[V] y puede generar picos de corriente durante su operación [?]. El LED necesita una resistencia limitadora para controlar y así mismo evitar el sobre consumo.
 - El sistema debe controlar tiempos específicos, como la duración de apertura del servo y el periodo de

bloqueo tras múltiples intentos fallidos. Por lo tanto, el funcionamiento depende de una gestión precisa del tiempo para garantizar un comportamiento adecuado.

- Las entradas mediante pulsadores pueden presentar rebote mecánico, lo que puede provocar lecturas inestables si no se considera esta condición en el comportamiento del sistema.

- **Variables Involucradas:** Para la implementación del algoritmo de control se definen diversas variables que permiten gestionar el funcionamiento del sistema. Entre ellas se encuentra un arreglo que almacena la contraseña correcta, otro arreglo que almacena la combinación ingresada por el usuario, una variable entera que registra el número de intentos fallidos y una constante que define el número máximo permitido antes de activar el bloqueo. Adicionalmente, se utilizan variables booleanas para indicar si el sistema se encuentra bloqueado y variables relacionadas con temporización para controlar la duración del bloqueo.

2) **Flujograma del sistema:** Se encuentra en la figura 2

3) *Justificación técnica del microcontrolador seleccionado:*

- **Número de pines requeridos:** El sistema requiere tres pines de entrada digital para los botones utilizados en la combinación de acceso. Adicionalmente, se requieren dos pines de salida para los LED indicadores de estado (acceso autorizado y acceso denegado) y un pin de salida adicional para el control del actuador de la cerradura (servo motor o relé). En total se requieren aproximadamente seis pines GPIO. El ESP32 dispone de más de 30 pines de entrada/salida programables, por lo que cumple ampliamente con este requerimiento.
- **Capacidad de procesamiento:** El microcontrolador cuenta con un procesador de doble núcleo basado en arquitectura Tensilica LX6 con frecuencia de operación de hasta 240 MHz, lo cual es suficiente para ejecutar la lógica de control del sistema, realizar la lectura de entradas digitales, gestionar temporizadores y generar señales PWM necesarias para el control de actuadores como servomotores. Su memoria RAM interna también permite implementar estructuras de datos para manejo de estados e intentos de autenticación.
- **Consumo energético:** El ESP32 presenta un consumo aproximado entre 80 mA y 240 mA en modo activo dependiendo de la carga de procesamiento, y puede reducir su consumo a valores del orden de microamperios cuando se utilizan modos de bajo consumo como deep sleep. Esto resulta adecuado para sistemas de control que permanecen largos periodos en estado de espera.
- **Esquema de conexión:** Se encuentra en la imagen 3

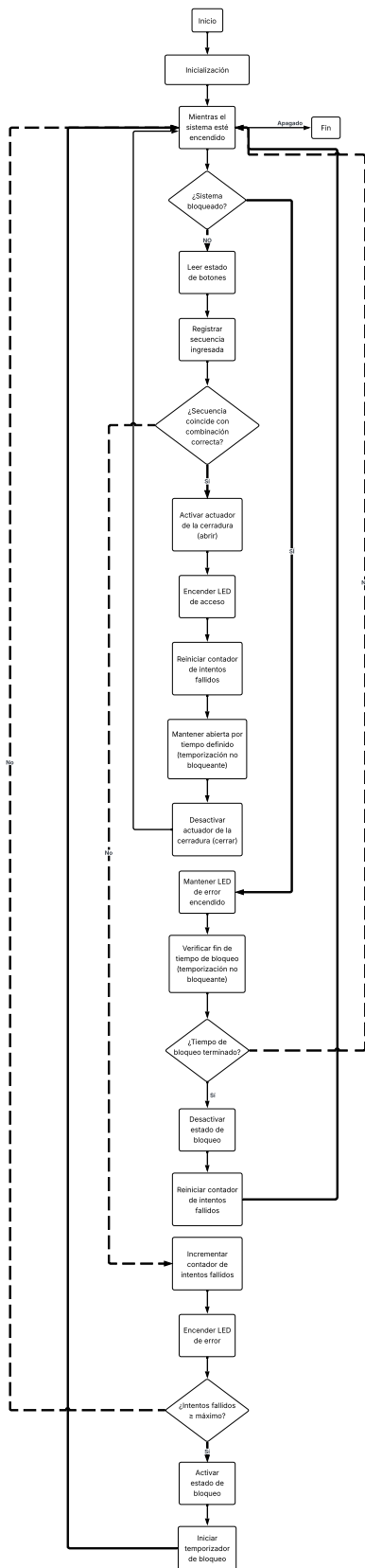


Figure 2. Flujograma reto 1

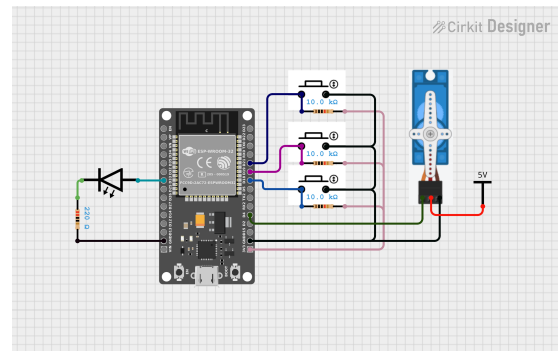


Figure 3. Diagrama de conexión reto 1

- **Componentes auxiliares** Para el correcto funcionamiento del sistema se requieren resistencias limitadoras de corriente para los LED (típicamente de 220 Ohm a 330 Ohm), resistencias de polarización para los botones (aproximadamente 10 kOhm), una fuente de alimentación estable para el microcontrolador y el actuador, y en caso de utilizar relé, un transistor driver con diodo de protección para evitar daños por voltajes inducidos. Estos componentes garantizan estabilidad eléctrica, protección de los dispositivos y operación confiable del sistema.

4) **Código documentado:** El código respectivo se encuentra en el anexo IX-A

B. Reto 2

Desarrollar un sistema que supervise una variable física y active alarmas según tres niveles de operación: normal, advertencia y crítico. Requisitos obligatorios:

- Definición clara de umbrales
- Implementación de histéresis para evitar inestabilidad
- Indicadores visuales o sonoros

1) Análisis del problema:

- **Identificación de entradas y salidas:** El sistema requiere una entrada correspondiente a la medición de una variable física que permita evaluar el estado del entorno supervisado. Para efectos de este diseño se selecciona la temperatura como variable de monitoreo, ya que constituye una de las variables más comunes en procesos industriales, sistemas electrónicos y aplicaciones de supervisión ambiental.

La medición de la temperatura será realizada mediante un sensor de temperatura analógico, el cual entrega una señal proporcional a la magnitud física medida. Esta señal será adquirida por el microcontrolador a través de uno de sus canales de conversión analógico-digital (ADC), permitiendo así transformar la señal analógica en un valor digital interpretable por el sistema.

En lo que corresponde a las salidas, el sistema contará con indicadores visuales y sonoros que permitirán identificar el estado operativo del sistema de monitoreo. Para ello se emplearán tres diodos LED que representarán los estados de operación normal, advertencia y condición crítica

respectivamente. De manera adicional se incorporará una alarma sonora mediante un buzzer, el cual se activará únicamente cuando el sistema detecte que la variable monitoreada ha alcanzado el nivel crítico.

• **Restricciones técnicas:**

- El sensor de temperatura seleccionado entrega una señal analógica que debe ser interpretada por el microcontrolador [?]. Por lo tanto, el dispositivo de procesamiento debe contar con un convertidor analógico–digital con suficiente resolución para permitir una medición adecuada de la variable física.
- Los diodos LED empleados como indicadores visuales requieren la inclusión de resistencias limitadoras de corriente, ya que los pines GPIO del microcontrolador presentan límites de corriente que no deben ser superados para evitar daños en el dispositivo.
- En el caso del buzzer o alarma sonora, es necesario considerar que algunos dispositivos pueden requerir corrientes superiores a las que puede suministrar directamente un pin del microcontrolador[?]. En estas condiciones puede ser necesario incorporar una etapa de conmutación basada en transistor, que permita manejar la carga sin comprometer la integridad del microcontrolador.
- Una restricción fundamental en este tipo de sistemas corresponde a la inestabilidad que puede presentarse cerca de los umbrales de decisión. Cuando la variable medida se encuentra próxima al valor límite entre dos estados, pequeñas fluctuaciones pueden provocar cambios continuos entre estados consecutivos. Para evitar este comportamiento se implementa un mecanismo de histéresis, el cual introduce un margen de diferencia entre los valores de activación y desactivación de cada estado [?].

- **Variables Involucradas:** Variable que almacena el valor digital leído del convertidor analógico–digital correspondiente al sensor de temperatura. Variables que definen los umbrales del sistema para los estados normal, advertencia y crítico. Variable que indica el estado actual del sistema para activar los indicadores correspondientes. Variable que establece el margen de histéresis para evitar cambios repetitivos entre estados. Variables auxiliares para controlar el tiempo de muestreo y la frecuencia de lectura del sensor.

2) **Flujograma del sistema:** Se encuentra en la figura 4.

3) **Justificación técnica del microcontrolador seleccionado:**

- **Número de pines requeridos.** El sistema requiere al menos un pin de entrada analógica para la lectura del sensor que mide la variable física supervisada. Adicionalmente, se requieren tres pines de salida digital para los indicadores de estado correspondientes a los niveles de operación: normal, advertencia y crítico. También puede incorporarse un pin adicional para activar una

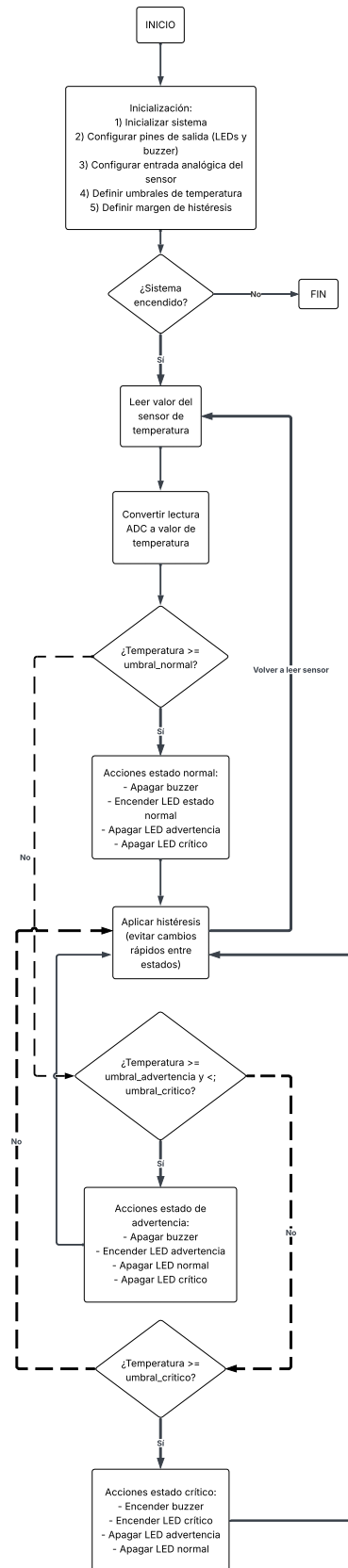


Figure 4. Flujograma reto 2

alarma sonora mediante un buzzer. En total, el sistema requiere aproximadamente cinco pines de entrada/salida. El microcontrolador ESP32 dispone de múltiples pines GPIO y varios canales ADC de 12 bits [?], lo que permite realizar la adquisición de la señal del sensor con suficiente resolución.

- **Capacidad de procesamiento.** El ESP32 cuenta con un procesador de doble núcleo con frecuencia de operación de hasta 240 MHz [?], lo cual permite ejecutar el algoritmo de monitoreo en tiempo real, realizar la conversión analógica-digital de la señal del sensor y aplicar la lógica de comparación con los umbrales definidos. Además, su capacidad de procesamiento permite implementar histéresis en los niveles de alarma para evitar conmutaciones rápidas e inestables cuando la variable medida se encuentra cercana a los valores límite [?].
- **Consumo energético.** El dispositivo presenta un consumo aproximado entre 80 mA y 240 mA en modo activo, dependiendo de la carga de procesamiento y los periféricos utilizados. Asimismo, dispone de modos de bajo consumo que reducen significativamente la corriente cuando el sistema se encuentra en espera. Esto permite su utilización en sistemas de monitoreo continuo sin requerir un consumo energético elevado.
- **Esquema de conexión.** Se encuentra en la imagen 5

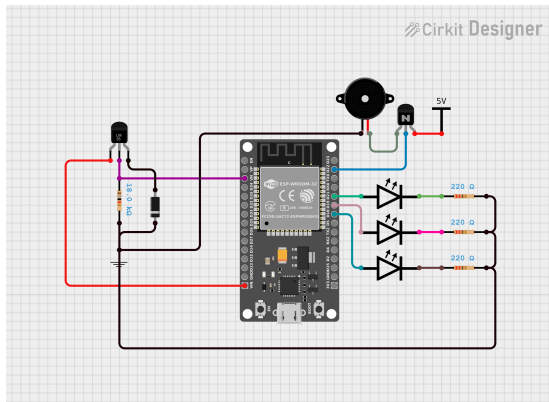


Figure 5. Diagrama de conexión reto 2

- **Componentes auxiliares** El sistema requiere resistencias limitadoras de corriente para los LED indicadores (220 Ohm a 330 Ohm), resistencias de polarización en caso de utilizar sensores analógicos que lo requieran, y una fuente de alimentación estable para el microcontrolador y los dispositivos de salida. Si se emplea un buzzer activo o una alarma de mayor potencia, puede ser necesario utilizar un transistor driver para manejar la corriente requerida.

4) **Código documentado:** El código respectivo se encuentra en el anexo IX-B

C. Reto 3

Diseñar un sistema de riego automático para cultivo de tomate. El sistema debe: Monitorear la humedad del suelo,

Activar riego cuando la humedad esté por debajo de un umbral, Detener el riego cuando se alcance nivel adecuado, Evitar inestabilidad mediante histéresis, Funcionar como sistema secuencial autónomo e Implemente temporización no bloqueante.

1) Análisis del problema:

- **Identificación de entradas y salidas:** La entrada principal es un sensor de humedad del suelo conectado a una entrada analógica del microcontrolador. La salida corresponde al actuador de riego (bomba o válvula solenoide) controlado mediante un relé o transistor de potencia. Opcionalmente se puede incluir un LED indicador de estado del sistema.

• Restricciones técnicas:

- El microcontrolador debe disponer de al menos una entrada analógica (ADC) para la lectura del sensor de humedad.
- Debe existir una salida digital capaz de controlar un relé o transistor para activar la bomba o válvula de riego.
- El sistema debe operar con niveles de voltaje compatibles (3.3 V o 5 V) entre microcontrolador, sensor y etapa de potencia. [?]
- El control del riego debe implementar histéresis mediante dos umbrales de humedad para evitar conmutaciones rápidas.
- El software debe emplear temporización no bloqueante (por ejemplo mediante contadores de tiempo) para permitir lecturas periódicas del sensor.
- **Variables involucradas:** Las variables principales son el valor de humedad del suelo medido por el sensor, el umbral inferior de activación del riego, el umbral superior de desactivación, el estado del sistema (riego activo o inactivo) y las variables de temporización utilizadas para el muestreo periódico del sensor.

2) **Flujograma del sistema :** Se encuentra en la figura 6

3) **Justificación técnica del microcontrolador seleccionado:**

- **Número de pines requeridos:** El sistema requiere 1 entrada analógica para el sensor de humedad del suelo y 1 salida digital para controlar el relé o transistor que activa la bomba de riego. Opcionalmente se puede utilizar 1 salida digital adicional para un LED indicador. El ESP32 dispone de más de 30 pines GPIO, incluyendo múltiples entradas ADC, por lo que satisface ampliamente los requerimientos del sistema.
- **Capacidad de procesamiento:** El ESP32 integra un procesador dual-core de hasta 240 MHz [?], suficiente para ejecutar el algoritmo de control secuencial, realizar lecturas periódicas del sensor mediante ADC, aplicar histéresis entre umbrales de humedad y gestionar temporización no bloqueante para el monitoreo continuo del sistema.
- **Consumo energético:** El microcontrolador opera a 3.3 V y presenta un consumo aproximado de 80–240 mA en

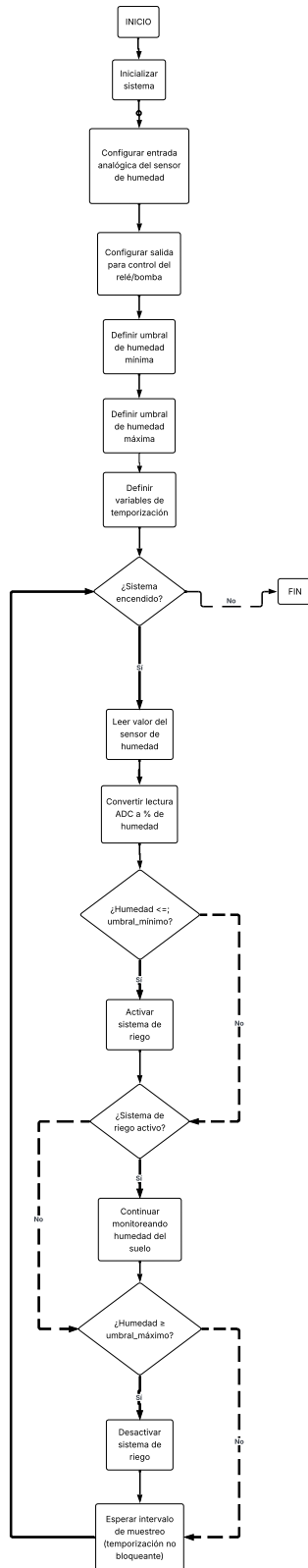


Figure 6. Flujograma reto 3

operación [?], con modos de bajo consumo disponibles. Esto permite su integración en sistemas agrícolas automatizados con alimentación estable o mediante fuentes reguladas.

- **Esquema de conexión (diagrama eléctrico o esquemático):** Se encuentra en la imagen 7

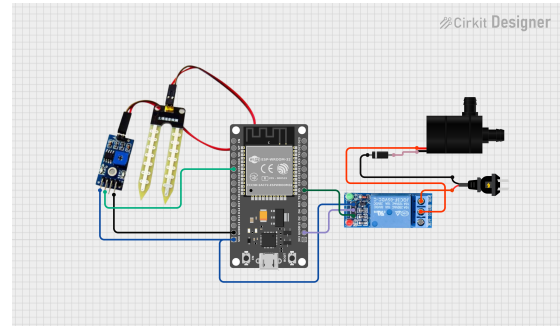


Figure 7. Diagrama de conexión reto 3

- **Componentes auxiliares** El sistema requiere un sensor de humedad del suelo, un módulo relé o MOSFET de potencia, resistencias de polarización para el control del transistor, diodo de protección, una bomba de agua o válvula solenoide para el riego y opcionalmente LEDs indicadores para mostrar el estado del sistema.

4) **Código documentado:** El código respectivo se encuentra en el anexo IX-C

IV. PROTOCOLOS DE COMUNICACIÓN

Los protocolos de comunicación permiten el intercambio de datos entre dispositivos dentro de un sistema embebido. Estos pueden ser de tipo serial o inalámbrico, dependiendo del medio de transmisión.

A. Comunicación Serial

La comunicación serial se basa en la transmisión secuencial de datos bit a bit a través de un canal, lo que permite reducir el cableado y mejorar la eficiencia del hardware.

- Transmisión secuencial de información
- Reducción del cableado
- Mayor eficiencia en el diseño
- Conexión entre microcontroladores y periféricos

Dentro de la comunicación serial se destacan los siguientes protocolos:

1) UART (Universal Asynchronous Receiver-Transmitter):

Es un protocolo de comunicación asíncrono que no utiliza señal de reloj compartida.

- Comunicación punto a punto
- Uso de tramas de datos
- Configuración mediante baudios, bits de datos, paridad y bits de parada
- Solo requiere dos líneas: TX y RX (más GND)

Ventajas:

- Simplicidad

- Bajo costo
- Comunicación full-duplex
- Mayor alcance comparado con otros protocolos seriales

2) **I2C (Inter-Integrated Circuit)**: Es un protocolo síncrono que permite la conexión de múltiples dispositivos utilizando solo dos líneas.

- Líneas: SDA (datos) y SCL (reloj)
- Permite múltiples dispositivos en el mismo bus
- Maneja direccionamiento (7 y 10 bits)
- Requiere resistencias pull-up

Ventajas:

- Reduce el número de conexiones
- Facilita la interconexión de varios periféricos

3) **SPI (Serial Peripheral Interface)**: Es un protocolo síncrono de alta velocidad basado en una arquitectura maestro-esclavo.

- Líneas: MOSI, MISO, SCK y SS/CS
- Comunicación full-duplex
- Alta velocidad de transmisión
- Sincronización mediante señal de reloj

Ventajas:

- Alta velocidad
- Precisión en la transmisión de datos

4) Comparación de Protocolos:

Característica	UART	I2C	SPI
Tipo	Asíncrono	Síncrono	Síncrono
Líneas	TX, RX	SDA, SCL	MOSI, MISO, SCK, SS
Reloj	No	Sí	Sí
Dispositivos	2	Múltiples	Maestro-esclavos
Velocidad	Media	Media	Alta
Complejidad	Baja	Media	Media

5) Criterios de Selección:

- Para alta velocidad: SPI
- Para múltiples dispositivos con pocas líneas: I2C
- Para comunicación simple punto a punto: UART

B. Comunicación Inalámbrica

Los protocolos inalámbricos son fundamentales en aplicaciones IoT y permiten la comunicación sin necesidad de conexiones físicas.

- Bluetooth Low Energy (BLE): bajo consumo y corto alcance
- Wi-Fi: alta tasa de datos e integración a internet
- Zigbee: redes malladas de bajo consumo
- Thread: comunicación en malla basada en IPv6
- LoRaWAN: largo alcance y bajo consumo
- NB-IoT: comunicación celular para IoT

1) Principales protocolos inalámbricos:

a) **Wi-Fi**: Protocolo diseñado inicialmente para redes de computadoras y acceso a internet. Actualmente es ampliamente utilizado en dispositivos IoT con suficiente disponibilidad energética.

- Alcance: ~100 m
- Velocidad: Alta (hasta Gbps)
- Uso: Cámaras, ESP32, interfaces HMI

b) **Bluetooth y Bluetooth Low Energy (BLE)**: Bluetooth es un estándar de corto alcance, mientras que BLE está optimizado para bajo consumo energético, ideal para dispositivos alimentados por batería.

- Alcance: 10–100 m
- Velocidad: Media (1–3 Mbps)
- Uso: Wearables, sensores cercanos

c) **ZigBee**: Protocolo orientado a redes de sensores distribuidos con bajo consumo energético, utilizando topologías malladas.

- Alcance: ~100 m
- Velocidad: Baja (250 kbps)
- Uso: Domótica, automatización industrial

d) **Thread**: Protocolo de red mallada basado en IPv6, diseñado para IoT, con capacidad de auto-reparación y bajo consumo energético.

- Red mallada segura
- Optimizado para dispositivos de baja potencia
- Uso: Hogares inteligentes y edificios

e) **LoRaWAN**: Protocolo de largo alcance y bajo consumo, ideal para aplicaciones donde los dispositivos están muy alejados entre sí.

- Alcance: 2–15 km
- Velocidad: Muy baja
- Uso: Agricultura, monitoreo ambiental, ciudades inteligentes

Tecnología	Alcance	Velocidad	Aplicación
Wi-Fi	~100 m	Alta	Internet, video, IoT con energía
Bluetooth/BLE	10–100 m	Media	Sensores, wearables
ZigBee	~100 m	Baja	Domótica
LoRaWAN	2–15 km	Muy baja	Monitoreo remoto

Table 1

COMPARACIÓN DE PROTOCOLOS INALÁMBRICOS

2) **Criterios de selección**: La elección del protocolo inalámbrico debe considerar:

- **Alcance**: Distancia entre dispositivos
- **Consumo energético**: Especialmente en dispositivos con batería
- **Velocidad de transmisión**: Cantidad de datos a enviar
- **Aplicación**: Tipo de sistema (industrial, doméstico, urbano)

En general:

- Wi-Fi: cuando se requiere alta velocidad
- BLE: cuando se necesita bajo consumo
- ZigBee/Thread: redes de múltiples dispositivos
- LoRaWAN: comunicación a larga distancia

3) **Importancia en IoT**: Los protocolos inalámbricos son la base del IoT, ya que permiten conectar sensores, dispositivos y sistemas a internet sin infraestructura cableada. Su correcta selección impacta directamente en el rendimiento, eficiencia y escalabilidad del sistema.

V. INTERNET TRADITIONAL VERSUS IOT

El internet tradicional se centra principalmente en la interacción entre usuarios y servicios digitales, donde el flujo de información incluye datos como texto, imágenes, audio y video. En este contexto, el usuario es el actor principal y el objetivo es el intercambio de información mediante plataformas web.

En contraste, el Internet de las Cosas (IoT) amplía este paradigma al integrar objetos físicos capaces de captar variables del entorno, procesarlas y comunicarlas a través de la red. El cambio fundamental radica en que el entorno físico se convierte en una fuente constante de datos útiles para monitoreo, control y toma de decisiones.

Por su parte, el Internet Industrial de las Cosas (IIoT) lleva este concepto al ámbito productivo, donde los datos se utilizan para optimizar procesos, mejorar la eficiencia, garantizar la seguridad y permitir la automatización avanzada en sistemas industriales.

A. Comparación conceptual

Aspecto	Internet Tradicional	IoT	IIoT
Actor principal	Usuarios y servicios web	Objetos + usuarios	Máquinas y procesos
Tipo de datos	Multimedia	Variables físicas	Datos de proceso
Objetivo	Intercambio de información	Monitoreo y control	Optimización y eficiencia
Ejemplo	Redes sociales	Casa inteligente	Industria conectada

B. IoT desde sistemas embebidos

Desde la perspectiva de sistemas embebidos, IoT se basa en la integración de cinco funciones:

- **Sensar:** capturar variables físicas (temperatura, presión, humedad, etc.)
- **Procesar:** filtrar y organizar los datos en el microcontrolador
- **Comunicar:** enviar información mediante protocolos y redes
- **Visualizar:** mostrar o almacenar los datos
- **Actuar:** ejecutar acciones sobre el sistema

Un sistema embebido deja de ser un dispositivo aislado y pasa a ser un nodo dentro de una arquitectura conectada.

C. Arquitectura básica IoT

Una solución IoT se estructura en capas:

- **Capa física:** sensores, actuadores y microcontroladores
- **Capa de comunicación:** redes y tecnologías de transmisión
- **Capa de servicio:** almacenamiento, procesamiento y gestión de datos
- **Capa de aplicación:** interacción con el usuario y toma de decisiones

El valor del sistema no está únicamente en el nodo embebido, sino en la capacidad de transformar datos en información útil.

D. Comunicación en IoT

Una solución IoT integra múltiples niveles de comunicación:

1) Dentro del nodo:

- GPIO, ADC y PWM
- Protocolos como I2C, SPI y UART

2) Hacia la red:

- Wi-Fi: alta tasa de datos
- BLE: bajo consumo
- LoRa: largo alcance
- Redes móviles: amplia cobertura

3) Protocolos de aplicación:

- HTTP: simple pero menos eficiente
- MQTT: ligero y eficiente para IoT
- WebSocket: comunicación en tiempo real

E. Lógica de implementación

El diseño de una solución IoT sigue una secuencia:

- 1) Definir la variable a medir
- 2) Seleccionar el sensor adecuado
- 3) Procesar los datos localmente
- 4) Transmitir la información
- 5) Almacenar o visualizar los datos
- 6) Generar una acción o decisión

F. De IoT a IIoT

El IIoT aplica estos principios en entornos industriales, permitiendo:

- Mantenimiento predictivo
- Monitoreo de procesos
- Optimización energética
- Trazabilidad y control de calidad

En este contexto, los datos se convierten en un recurso clave para la toma de decisiones en tiempo real.

G. Consideraciones de diseño

- Seguridad de la información
- Consumo energético
- Robustez del sistema
- Escalabilidad
- Mantenibilidad

VI. PROYECTO FINAL DE SISTEMAS EMBEBIDOS

A. Descripción del problema

En pequeños cultivos de hortalizas el riego y el control de condiciones ambientales suelen realizarse de forma manual, lo que provoca uso ineficiente del agua, variaciones en la humedad del sustrato, estrés hídrico en las plantas y baja trazabilidad del proceso. Se propone un sistema embebido IoT que automatice el riego por goteo y la regulación térmica, registre telemetría y permita visualización remota para mejorar eficiencia y reproducibilidad experimental.

B. Objetivo general y objetivos específicos

Objetivo general: Diseñar e implementar un sistema embebido IoT para el monitoreo remoto y control automático de variables asociadas al crecimiento de la hortaliza asignada por la profesora, integrando sensores, actuadores, comunicación inalámbrica y plataforma de visualización.

Objetivos específicos:

- 1) Definir requisitos funcionales y no funcionales del sistema considerando la hortaliza asignada.
- 2) Seleccionar y justificar la tarjeta de desarrollo, sensores y actuadores adecuados.
- 3) Diseñar la arquitectura hardware/software y la partición de responsabilidades.
- 4) Implementar firmware modular con control de riego y control térmico con histéresis.
- 5) Integrar comunicación IoT (Firebase o MQTT) para publicación y control remoto.
- 6) Validar el prototipo mediante pruebas experimentales y documentar resultados en GitHub y bitácora.

C. Requisitos funcionales

- 1) Medir al menos tres variables: **temperatura ambiente**, **humedad relativa del aire** y **humedad del sustrato**.
- 2) Controlar al menos dos actuadores: **bomba de riego** y **actuador térmico** (ventilador o calefactor).
- 3) Implementar control automático de riego basado en umbral de humedad del sustrato con histéresis.
- 4) Implementar control térmico basado en umbral de temperatura con histéresis.
- 5) Publicar telemetría periódica y estados de actuadores en una plataforma IoT (Firebase o broker MQTT).
- 6) Permitir recepción de comandos remotos (p. ej. forzar riego) mediante tópicos MQTT o escritura en Firebase.
- 7) Registrar eventos y estados en memoria no volátil (logs mínimos) y evidenciar commits en GitHub.

D. Requisitos no funcionales

- **Desarrollo:** usar Visual Studio Code; no usar Arduino IDE; evitar archivos .ino.
- **Calidad del firmware:** modularidad, nombres descriptivos, comentarios técnicos, evitar `delay()` en la lógica principal.
- **Conectividad:** conexión Wi-Fi estable para sincronización periódica; tolerancia a desconexiones temporales (buffer local).
- **Seguridad:** credenciales Wi-Fi y credenciales IoT almacenadas de forma segura; comunicación MQTT con autenticación si es posible.
- **Energía:** consumo razonable; diseño que permita alimentación por fuente DC estable.
- **Escalabilidad:** arquitectura que permita añadir sensores/actuadores sin reescritura masiva.
- **Documentación:** README claro, diagrama de conexiones, bitácora con evidencias y commits frecuentes.

E. Diagrama general del sistema

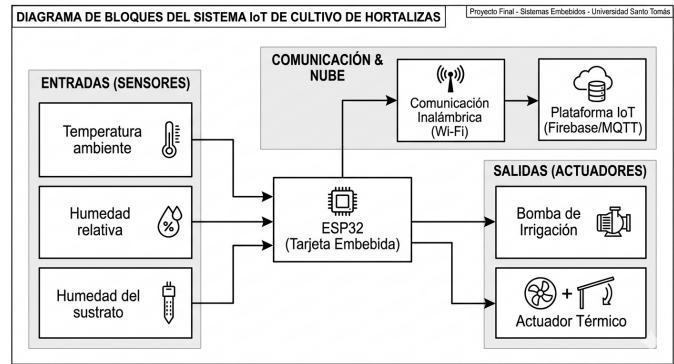


Figure 8. Diagrama general del sistema

F. Selección justificada de la tarjeta

Tarjeta recomendada: ESP32 (familia ESP32-WROOM / ESP32-S2).

Justificación técnica:

- **Conectividad integrada:** Wi-Fi (y Bluetooth) incorporado, facilita comunicación con Firebase o broker MQTT sin módulos adicionales.
- **Entradas analógicas:** múltiples ADCs suficientes para sensores de humedad de sustrato y otros sensores analógicos.
- **Capacidad de procesamiento:** MCU dual-core con frecuencia suficiente para tareas de adquisición, control y comunicación simultánea.
- **Ecosistema y librerías:** amplia disponibilidad de librerías (MQTT, Firebase, sensores) y soporte en PlatformIO/VSCode.
- **GPIO y periféricos:** suficientes pines digitales para relés, drivers y sensores digitales.
- **Consumo y coste:** balance adecuado entre consumo, coste y facilidad de uso para proyectos académicos.

G. Listado de sensores y actuadores (propuesta)

Sensores:

- **Sensor de temperatura ambiente:** SHT31 (recomendado por precisión).
- **Sensor de humedad relativa del aire:** integrado en SHT31 o DHT22.
- **Sensor de humedad del sustrato:** sensor capacitivo de humedad de suelo (salida analógica).
- **Sensor de luminosidad (opcional):** BH1750 o LDR con divisor.
- **Sensor de nivel de agua (opcional):** flotador o sensor ultrasónico para tanque.

Actuadores:

- **Bomba de riego** controlada por relé o driver MOSFET.
- **Válvula solenoide** para riego por zonas (opcional).
- **Actuador térmico:** ventilador DC o resistencia calefactora con driver.
- **Motor/servomotor** para techo corredizo (si aplica).

- **Indicadores locales:** LEDs de estado, buzzer para alarmas.

H. Partición hardware / software

Funciones en hardware:

- Lectura física de sensores (ADC, I2C).
- Acondicionamiento de señales (filtros RC, divisores).
- Protección eléctrica y drivers para actuadores (relés, MOSFETs).
- Fuente de alimentación estable y protección (fusibles).

Funciones en software (firmware):

- Lectura periódica y validación de sensores.
- Filtrado digital básico (media móvil, mediana).
- Lógica de control con histéresis para riego y control térmico.
- Publicación periódica de telemetría en Firebase o tópicos MQTT (JSON).
- Suscripción a comandos remotos y ejecución segura de órdenes.
- Gestión de estados, alarmas y registro de eventos en memoria.
- Mecanismos de reconexión y buffer local en caso de pérdida de conectividad.

I. Diagrama de flujo o máquina de estados

DIAGRAMA DE FLUJO DE FIRMWARE IoT EMBEBIDO

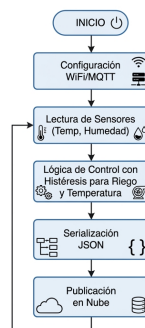


Figure 9. Diagrama general de flujo del sistema

J. Parámetros recomendados iniciales y pruebas

- **Humedad sustrato:** definir humedad_min según la hortaliza asignada (ej. 40% para lechuga en sustrato ligero) y histeresis = 5%.
- **Temperatura:** temp_max según especie (ej. 28–30 °C); histeresis_temp = 2°C.
- **Frecuencia de muestreo:** lectura cada 10–30 s; publicación IoT cada 30–120 s.
- **Pruebas:** calibrar sensor de humedad de sustrato con medidas gravimétricas; verificar tiempos de riego y consumo de agua; validar reconexión Wi-Fi y buffer local.

K. Entregables técnicos asociados a este documento

- Documento de requisitos y diseño (este archivo LaTeX).
- Diagrama de conexiones (archivo PNG/SVG en carpeta hardware/) (a completar).

- README con instrucciones de compilación y ejecución (PlatformIO + VSCode).
- Plantilla de configuración IoT (ejemplo de firebase_config_example.json o tópicos MQTT).

VII. RESULTADOS

VIII. CONCLUSIONES

En esta plantilla:

- Se mostró la estructura básica de un artículo IEEEtran en LaTeX en español. - Se incluyeron ejemplos mínimos de figuras, tablas, ecuaciones y citas. - Se proporcionaron comentarios en el código para guiar el uso en clase.

Trabajo futuro: reemplazar el contenido de ejemplo por el de tu proyecto, añadir más secciones si es necesario y ajustar el preámbulo según las normas de la conferencia o revista.

AGRADECIMIENTOS

Se agradece a la institución X por el apoyo y a la agencia Y por la financiación del proyecto Z. (Sustituir por información real o eliminar esta sección si no aplica.)

IX. ANEXOS

A. Código Reto 1

```

//Microcontrolador: ESP32//

//DEFINICIÓN DE PINES -//

const int boton1 = 25;
const int boton2 = 26;
const int boton3 = 27;

const int ledAcceso = 14;
const int ledError = 12;

const int releCerradura = 13;

//PARÁMETROS DEL SISTEMA //

const int secuenciaCorrecta[3] = {1,2,3};

const int maxIntentos = 3;

const unsigned long tiempoBloqueo = 10000;
const unsigned long tiempoApertura = 3000;

//VARIABLES DE CONTROL //

int secuenciaIngresada[3];
int indiceSecuencia = 0;

int contadorIntentos = 0;

bool sistemaBloqueado = false;

unsigned long inicioBloqueo = 0;
unsigned long inicioApertura = 0;

bool cerraduraAbierta = false;

//CONFIGURACIÓN //

void setup()
{
    pinMode(boton1, INPUT_PULLUP);
    pinMode(boton2, INPUT_PULLUP);
    pinMode(boton3, INPUT_PULLUP);

    pinMode(ledAcceso, OUTPUT);
    pinMode(ledError, OUTPUT);

    pinMode(releCerradura, OUTPUT);

    Serial.begin(115200);
}

//LOOP PRINCIPAL //

void loop()
{
    verificarBloqueo();
  
```

```

    if(!sistemaBloqueado)
    {
        leerBotones();
        verificarSecuencia();
    }

    controlarApertura();
}

//LECTURA DE BOTONES //

void leerBotones()
{
    if(digitalRead(boton1)==LOW)
    {
        registrarEntrada(1);
    }

    if(digitalRead(boton2)==LOW)
    {
        registrarEntrada(2);
    }

    if(digitalRead(boton3)==LOW)
    {
        registrarEntrada(3);
    }
}

// REGISTRAR ENTRADA //

void registrarEntrada(int valor)
{
    if(indiceSecuencia < 3)
    {
        secuenciaIngresada[indiceSecuencia] = valor;
        indiceSecuencia++;
    }
}

// VERIFICAR COMBINACION //

void verificarSecuencia()
{
    if(indiceSecuencia == 3)
    {
        if(combinacionCorrecta())
        {
            abrirCerradura();
            contadorIntentos = 0;
        }
        else
        {
            manejarIntentoFallido();
        }

        indiceSecuencia = 0;
    }
}

// COMPARACION //

bool combinacionCorrecta()
{
    for(int i=0;i<3;i++)
    {
        if(secuenciaIngresada[i] != secuenciaCorrecta[i])
            return false;
    }

    return true;
}

//APERTURA//

void abrirCerradura()
{
    digitalWrite(releCerradura, HIGH);
    digitalWrite(ledAcceso, HIGH);

    cerraduraAbierta = true;
    inicioApertura = millis();
}

//CONTROL APERTURA //

void controlarApertura()
{
    if(cerraduraAbierta)
    {
        if(millis() - inicioApertura > tiempoApertura)
        {
            digitalWrite(releCerradura, LOW);
            digitalWrite(ledAcceso, LOW);
            cerraduraAbierta = false;
        }
    }
}

//INTENTO FALLIDO//

void manejarIntentoFallido()
{

```

```

    contadorIntentos++;

    digitalWrite(ledError, HIGH);

    if(contadorIntentos >= maxIntentos)
    {
        sistemaBloqueado = true;
        inicioBloqueo = millis();
    }
}

// CONTROL BLOQUEO //

void verificarBloqueo()
{
    if(sistemaBloqueado)
    {
        if(millis() - inicioBloqueo > tiempoBloqueo)
        {
            sistemaBloqueado = false;
            contadorIntentos = 0;
            digitalWrite(ledError, LOW);
        }
    }
}

```

B. Código Reto 2

```

const int sensorPin = 34;

const int ledNormal = 16;
const int ledAdvertencia = 17;
const int ledCritico = 18;

const int buzzer = 19;

/* Definición de umbrales del sistema */

float V1 = 30;
float V2 = 40;

float H = 2;

/* Definición de estados del sistema */

enum EstadoSistema
{
    NORMAL,
    ADVERTENCIA,
    CRITICO
};

EstadoSistema estadoActual = NORMAL;

/* Configuración inicial del sistema */

void setup()
{
    pinMode(ledNormal,OUTPUT);
    pinMode(ledAdvertencia,OUTPUT);
    pinMode(ledCritico,OUTPUT);

    pinMode(buzzer,OUTPUT);

    Serial.begin(115200);
}

/* Bucle principal de ejecución */

void loop()
{
    float valorSensor = leerSensor();

    actualizarEstado(valorSensor);

    actualizarSalidas();
}

/* Lectura del sensor analógico */

float leerSensor()
{
    int lecturaADC = analogRead(sensorPin);

    float temperatura = (lecturaADC/4095.0)*100;

    return temperatura;
}

/* Actualización del estado del sistema con histograma */

void actualizarEstado(float valor)
{
    switch(estadoActual)
    {
        case NORMAL:
            if(valor > V1 + H)
                estadoActual = ADVERTENCIA;

```

```

        break;

    case ADVERTENCIA:

        if(valor > V2 + H)
            estadoActual = CRITICO;

        if(valor < V1 - H)
            estadoActual = NORMAL;

        break;

    case CRITICO:

        if(valor < V2 - H)
            estadoActual = ADVERTENCIA;

        break;

    }
}

/* Control de salidas del sistema */

void actualizarSalidas()
{
    digitalWrite(ledNormal, LOW);
    digitalWrite(ledAdvertencia, LOW);
    digitalWrite(ledCritico, LOW);

    switch(estadoActual)
    {

        case NORMAL:

            digitalWrite(ledNormal, HIGH);

            break;

        case ADVERTENCIA:

            digitalWrite(ledAdvertencia, HIGH);

            break;

        case CRITICO:

            digitalWrite(ledCritico, HIGH);
            digitalWrite(buzzer, HIGH);

            break;

    }
}

```

C. Código Reto 3

```

const int sensorHumedad = 35;

const int releBomba = 23;

/* Definición de parámetros de control */

int umbralRiego = 40;

int hist = 5;

/* Parámetros de temporización del riego */

unsigned long tiempoRiego = 8000;

unsigned long inicioRiego = 0;

bool bombaActiva = false;

/* Configuración inicial del sistema */

void setup()
{
    pinMode(releBomba, OUTPUT);
    Serial.begin(115200);
}

/* Bucle principal de ejecución */

void loop()
{
    int humedad = leerHumedad();

    controlRiego(humedad);

    controlarTemporizador();
}

/* Lectura del sensor de humedad */

```

```

int leerHumedad()
{
    int lectura = analogRead(sensorHumedad);

    int porcentaje = map(lectura, 0, 4095, 100, 0);

    return porcentaje;
}

/* Lógica de control para iniciar el riego */

void controlRiego(int humedad)
{
    if(!bombaActiva)
    {
        if(humedad < (umbralRiego - hist))
        {
            iniciarRiego();
        }
    }
}

/* Activación del sistema de riego */

void iniciarRiego()
{
    digitalWrite(releBomba, HIGH);

    bombaActiva = true;

    inicioRiego = millis();
}

/* Control del temporizador de riego */

void controlarTemporizador()
{
    if(bombaActiva)
    {
        if(millis() - inicioRiego > tiempoRiego)
        {
            detenerRiego();
        }
    }
}

/* Desactivación del sistema de riego */

void detenerRiego()
{
    digitalWrite(releBomba, LOW);

    bombaActiva = false;
}

```